

CREDIFAB

DOCUMENTO DE ARQUITETURA

Versão 1.2

3 de julho de 2026

Tabela 1 – Integrantes do Grupo

Matrícula	Nome	Papéis	Pontos
241025499	Alan Semil dos Santos Vieira	Desenvolvedor Backend	8,34
232024509	Anna Júlia Aparecida Silva Primo	Desenvolvedora Frontend	8,34
242015782	Cibelle de Assis Silva	Desenvolvedora Frontend	8,34
251019771	Daniel Filipe Borges de Oliveira	Analista de Qualidade e Testador de Software	8,34
251008114	Eduardo Jesus Dal Pizzol	Líder do Grupo; Arquiteto/Analista de Banco de Dados e Domínio	8,34
251009185	Gabriel Melo Rodrigues Cardone	Desenvolvedor Backend	8,34
251010186	Igor Dantas Araújo	Desenvolvedor Backend	8,34
242028708	João Marcos Santos e Carvalho	Desenvolvedor Frontend	8,34
251023184	João Pedro da Nóbrega Souza Cordeiro	Desenvolvedor Backend	8,34
242015871	Júlia Amanda Silva Lima	Desenvolvedora Frontend	8,34
242028842	Maria Eduarda de Oliveira Gomes	Arquiteta/Analista de Banco de Dados e Domínio	8,34
251013660	Matheus Moretti Soares	Analista de Qualidade e Testador de Software	8,34

Fonte: elaborado pelo autor (2026).

**Os pontos de contribuição foram distribuídos igualmente entre todos os integrantes, pois todos foram autores deste documento.*

Tabela 2 – Histórico de Revisões

Data	Versão	Descrição	Autor(es)
08/05/2026	0.1	Versão inicial com o detalhamento arquitetural, escopo e bibliografias.	Daniel e João Pedro.
09/05/2026	1.0	Versão final para a primeira entrega do Documento de Arquitetura.	Todos os integrantes do grupo.
28/05/2026	1.1	Correção de erros, padronização e maior adequação ao Documento de Visão.	Daniel.
03/07/2026	1.2	Correção Deploy	Daniel.

Fonte: elaborado pelo autor (2026).

Sumário

Integrantes do Grupo	1
Histórico de Revisões	2
1 Introdução	4
1.1 Propósito	4
1.2 Escopo	4
2 Representação Arquitetural	4
2.1 Definições	4
2.2 Justificativa da escolha	4
2.3 Detalhamento	5
2.3.1 Explicação do Diagrama PlantUML em camadas	6
2.4 Metas e restrições arquiteturais	7
2.4.1 Metas Arquiteturais	7
2.4.2 Restrições Arquiteturais	9
2.5 Visões	11
2.5.1 Visão de uso	11
2.5.2 Visão de organização lógica	14
2.5.3 Visão estrutural	15
2.6 Visão de Implantação	18
2.7 Restrições adicionais	19
3 Bibliografia	20

1 Introdução

1.1 Propósito

Este documento descreve a arquitetura do sistema sendo desenvolvido pelo grupo, na disciplina de MDS – Métodos de Desenvolvimento de Software – edição do primeiro semestre de 2026, para o sistema CREDIFAB, a fim de fornecer uma visão abrangente do sistema para desenvolvedores, testadores e demais interessados em aspectos relacionados às tecnologias a serem usadas no desenvolvimento.

1.2 Escopo

O detalhamento do escopo se encontra no Documento de Visão de Produto e do Projeto. Porém, em linhas gerais, o escopo do produto compreende as funcionalidades do sistema, classificadas por prioridade: Must (essencial), Should (importante) e Could (desejável). Além do nosso Perfil de Usuário, cujo foco se dá em gestores/proprietários, com permissão para fornecer seus dados financeiros, anexar documentos, gerar relatórios e realizar simulações de crédito. Por fim, abarca os cenários funcionais (sprints) e os requisitos de produto do nosso cronograma.

2 Representação Arquitetural

2.1 Definições

O sistema seguirá o estilo arquitetural Cliente-Servidor (Client-Server), estruturado em múltiplos níveis (multi-tier), com uma separação lógica e física bem definida entre a interface de usuário (Frontend), a lógica de negócio e serviços (Backend) e a persistência de dados (Banco de Dados). A comunicação entre a aplicação cliente e o servidor será realizada por meio de uma API RESTful, garantindo o baixo acoplamento entre as camadas e padronizando a troca de informações.

2.2 Justificativa da escolha

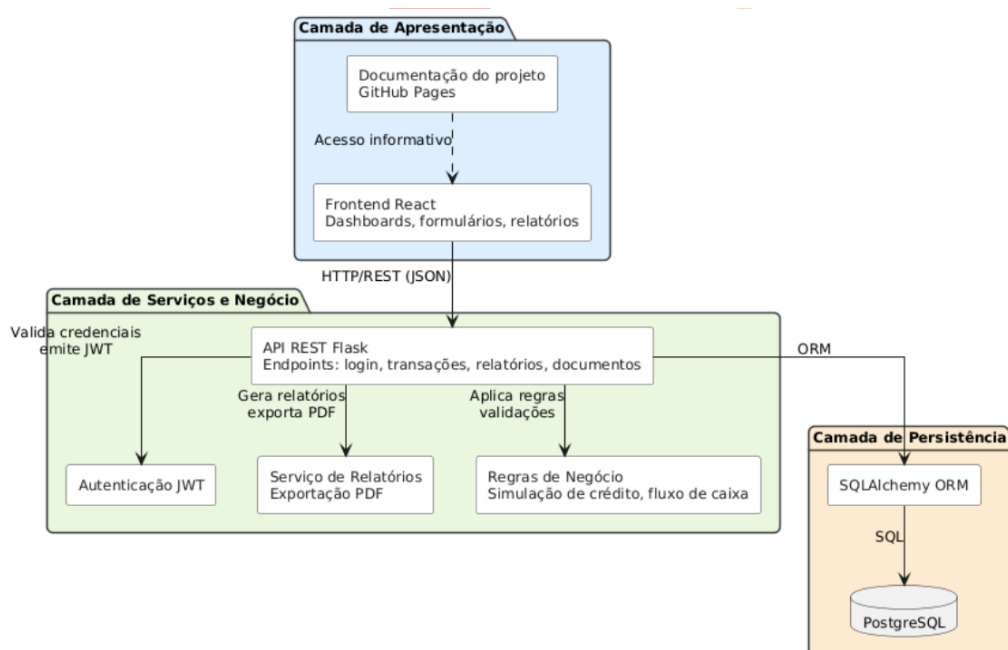
A escolha pelo estilo arquitetural Cliente-Servidor com comunicação via API RESTful é justificada pelas necessidades operacionais e de organização do projeto CREDIFAB, conforme evidenciado no Documento de Visão:

- **Separação de Responsabilidades e Desenvolvimento Paralelo:** O projeto possui uma estrutura de equipe com papéis estritamente delineados entre “Desenvolvedor Backend” e “Desenvolvedor Frontend”. A adoção de uma API RESTful permite que as equipes de interface (React) e de serviços (Python/Flask) trabalhem em paralelo de forma isolada, definindo previamente os contratos da API para mitigar riscos de integração.
- **Estratégia de Implantação Desacoplada:** O Documento de Visão estabelece que o sistema será uma aplicação web com ambientes de hospedagem distintos: o Frontend será hospedado no GitHub Pages, enquanto o Backend será hospedado na nuvem Render. Esse modelo exige uma arquitetura distribuída, em que o cliente consuma os serviços do servidor remotamente.
- **Segurança e Isolamento de Dados:** Como o domínio envolve governança financeira e upload de documentos sensíveis das MPES, a arquitetura Cliente-Servidor garante que as regras de negócio e a comunicação com o Banco de Dados (PostgreSQL via SQLAlchemy) fiquem isoladas no servidor. Além disso, a autenticação baseada em tokens JWT citada nos requisitos adapta-se nativamente ao caráter stateless das APIs REST.

2.3 Detalhamento

O sistema CREDIFAB seguirá a arquitetura Cliente-Servidor em múltiplos níveis, com a separação lógica entre as camadas de Apresentação, Serviços/Negócios e Persistência, garantindo o baixo acoplamento e organização clara das responsabilidades. A imagem abaixo apresenta o diagrama da Arquitetura de Software do CREDIFAB, que foi desenvolvido através do site <https://www.plantuml.com>, acessado em 08/05/2026.

Figura 1: Diagrama PlantUML de Arquitetura de Software.



Fonte: elaborado pelos autores (2026).

2.3.1 Explicação do Diagrama PlantUML em camadas

Apresentação: Será composta pelo Frontend React, responsável pela interface com o usuário, exibindo dashboards, formulários e relatórios. Essa camada comunica-se com a Camada de Serviços por meio de chamadas HTTP/REST (JSON) e não contém regras de negócio críticas. A documentação do projeto, hospedada no GitHub Pages, tem função apenas informativa e não participa do fluxo operacional do sistema. **Serviços e Negócio:** Implementada no Backend em Flask, que expõe a API REST com os endpoints responsáveis por autenticação, categorias, transações financeiras, documentos, relatórios e simulação de crédito. Nesta camada são aplicadas as regras de negócio, incluindo cálculos de fluxo de caixa e simulação de crédito. A autenticação será feita por JWT, garantindo sessões seguras compatíveis com o modelo stateless das APIs REST. A geração de relatórios exportáveis em PDF será responsabilidade de um serviço específico nesta camada. **Persistência:** Composta pelo PostgreSQL, acessado por meio do ORM SQLAlchemy, responsável por armazenar dados de usuários, empresas, categorias, transações, documentos e simulações de crédito. O uso do ORM padroniza o acesso aos dados, reduzindo acoplamento e garantindo integridade.

2.4 Metas e restrições arquiteturais

As metas e restrições arquiteturais do CREDIFAB foram definidas considerando o contexto do sistema, os requisitos funcionais levantados no Documento de Visão, as tecnologias adotadas pela equipe e o perfil dos usuários da aplicação. O objetivo dessas diretrizes é garantir que a arquitetura do sistema seja segura, organizada, escalável e adequada às necessidades das micro e pequenas empresas que utilizarão a plataforma.

2.4.1 Metas Arquiteturais

As metas e restrições arquiteturais do sistema CREDIFAB foram definidas com base nos requisitos funcionais apresentados no Documento de Visão do Produto e Projeto, nas tecnologias escolhidas pela equipe e no contexto acadêmico de desenvolvimento da aplicação. Essas diretrizes têm como objetivo garantir segurança, organização, modularidade, facilidade de manutenção, usabilidade e qualidade geral do sistema ao longo de seu desenvolvimento incremental.

2.4.1.1 Garantir modularidade e baixo acoplamento. A arquitetura do sistema deverá manter separação clara entre interface, regras de negócio e persistência de dados, reduzindo o acoplamento entre os componentes da aplicação. A adoção da arquitetura Cliente-Servidor com comunicação via API RESTful permitirá que frontend, backend e banco de dados evoluam de forma independente, facilitando manutenção, testes e implementação de novas funcionalidades ao longo das Sprints. Essa meta permitirá que funcionalidades sejam desenvolvidas, testadas e mantidas de forma independente, facilitando a evolução incremental do sistema ao longo das Sprints.

2.4.1.2 Garantir segurança no armazenamento de dados financeiros. O sistema deverá proteger os dados financeiros e documentais das empresas cadastradas, garantindo que apenas usuários autenticados possam acessar informações da própria organização. Para isso, a arquitetura utilizará autenticação baseada em JWT, separação lógica dos dados e controle de acesso às rotas protegidas da API. Essa meta é importante porque o sistema manipula informações empresariais sensíveis, como fluxo de caixa, relatórios financeiros e documentos utilizados em processos de crédito.

2.4.1.3 Garantir usabilidade e simplicidade operacional. A aplicação deverá possuir interface intuitiva, organizada e de fácil utilização, permitindo que gestores de micro e pequenas empresas utilizem o sistema mesmo sem conhecimentos técnicos avançados em tecnologia ou gestão financeira. A arquitetura do frontend deverá priorizar:

- navegação simplificada;
- organização clara das funcionalidades;
- padronização visual das telas;
- redução da complexidade operacional;
- feedback visual adequado durante as ações do usuário.

Essa meta é importante porque o público-alvo do CREDIFAB inclui pequenos empresários que frequentemente possuem dificuldades com organização financeira e ferramentas digitais complexas.

2.4.1.4 Garantir facilidade de manutenção e evolução do sistema. O sistema deverá possuir estrutura organizada e modular para facilitar manutenção, correção de falhas e evolução contínua da aplicação. Para isso:

- o backend será organizado em módulos separados por responsabilidade;
- o frontend utilizará componentização em React;
- as regras de negócio ficarão isoladas da camada de apresentação;
- a equipe seguirá padrões de codificação e revisão contínua de código.

Essa meta contribui para reduzir complexidade, facilitar entendimento do código e melhorar a produtividade da equipe durante o desenvolvimento incremental do projeto.

2.4.1.5 Garantir qualidade contínua do software. A arquitetura deverá permitir execução automatizada de testes e integração contínua ao longo do desenvolvimento, em conformidade com a estratégia de testes definida no Documento de Visão. O projeto utilizará:

- testes unitários (Pytest no backend; Vitest e React Testing Library no frontend);
- testes de integração (rotas da API contra o banco; integração frontend-API);
- testes de sistema (E2E) com Playwright, executados pela dupla de Qualidade em ambiente local nas branches `test/`;
- pipeline automatizada no GitHub Actions executando testes unitários e de integração a cada Pull Request para a branch `develop`.

2.4.2 Restrições Arquiteturais

As restrições arquiteturais do sistema CREDIFAB definem limitações, padrões e diretrizes técnicas que deverão ser seguidos durante o desenvolvimento da aplicação. Essas restrições têm como objetivo garantir padronização, organização, compatibilidade entre os componentes do sistema e alinhamento com as tecnologias e metodologias adotadas pela equipe.

2.4.2.1 Restrição tecnológica. O projeto deverá utilizar exclusivamente as tecnologias definidas pela equipe no Documento de Visão:

- React no frontend;
- Python com Flask no backend;
- PostgreSQL com SQLAlchemy na persistência;
- Vercel e Render para hospedagem.

Essa restrição garante padronização tecnológica e compatibilidade entre os componentes do sistema.

2.4.2.2 Restrição de estilo arquitetural. O sistema deverá seguir obrigatoriamente o modelo arquitetural Cliente-Servidor em múltiplas camadas. A comunicação entre frontend e backend deverá ocorrer por meio de uma API RESTful, permitindo desacoplamento entre interface, regras de negócio e persistência de dados. Além disso, a autenticação da aplicação deverá utilizar tokens JWT para controle de acesso às funcionalidades protegidas do sistema. Essa restrição contribui para modularidade, manutenção e escalabilidade da aplicação.

2.4.2.3 Restrição de padronização de código. O backend deverá seguir o padrão PEP8 para desenvolvimento Python. Além disso:

- variáveis utilizarão snake_case;
- classes utilizarão PascalCase;
- endpoints REST seguirão nomenclatura padronizada;
- funções críticas possuirão docstrings.

No frontend:

- componentes React deverão possuir responsabilidade única;
- arquivos serão organizados de forma modular e reutilizável.

Linters e revisões de código serão utilizados para validação dessas regras.

2.4.2.4 Restrição de versionamento e integração. O desenvolvimento deverá seguir a estratégia Git Flow definida pela equipe. As branches serão organizadas em:

- main;
- develop;
- feature/*;
- test/*;
- fix/*.

Além disso:

- merges ocorrerão apenas via Pull Request;
- alterações deverão passar por revisão antes da integração;
- funcionalidades deverão ser validadas por testes antes de serem integradas às branches principais.

Essa restrição garante rastreabilidade, estabilidade e controle de qualidade durante o desenvolvimento colaborativo.

2.4.2.5 Restrição operacional e acadêmica. Por se tratar de um projeto acadêmico:

- o sistema utilizará serviços gratuitos de hospedagem;
- os testes utilizarão dados fictícios;
- o desenvolvimento ocorrerá de forma incremental conforme o cronograma das Sprints;
- as funcionalidades serão implementadas conforme priorização do backlog.

Essa restrição reduz custos, limita riscos relacionados a dados reais e mantém compatibilidade com o contexto da disciplina.

2.5 Visões

Esta seção tem por objetivo apresentar as visões de uso, de organização lógica, estrutural e de implementação do sistema CREDIFAB. Por meio dos diagramas e descrições apresentados, é possível compreender como os usuários interagem com o sistema e como os requisitos do projeto influenciaram as decisões arquiteturais adotadas pela equipe.

2.5.1 Visão de uso

A aplicação web CREDIFAB foi desenvolvida para atender ao Objetivo de Desenvolvimento Sustentável 9, com foco em micro e pequenas empresas na organização de informações financeiras e documentais, o que permite um planejamento melhor para a garantia de crédito em condições adequadas à realidade dessas instituições. O sistema oferece funcionalidades de registro financeiro, geração de relatórios, controle de fluxo de caixa, gerenciamento documental e simulação de crédito, apoiando a tomada de decisão e promovendo maior transparência financeira.

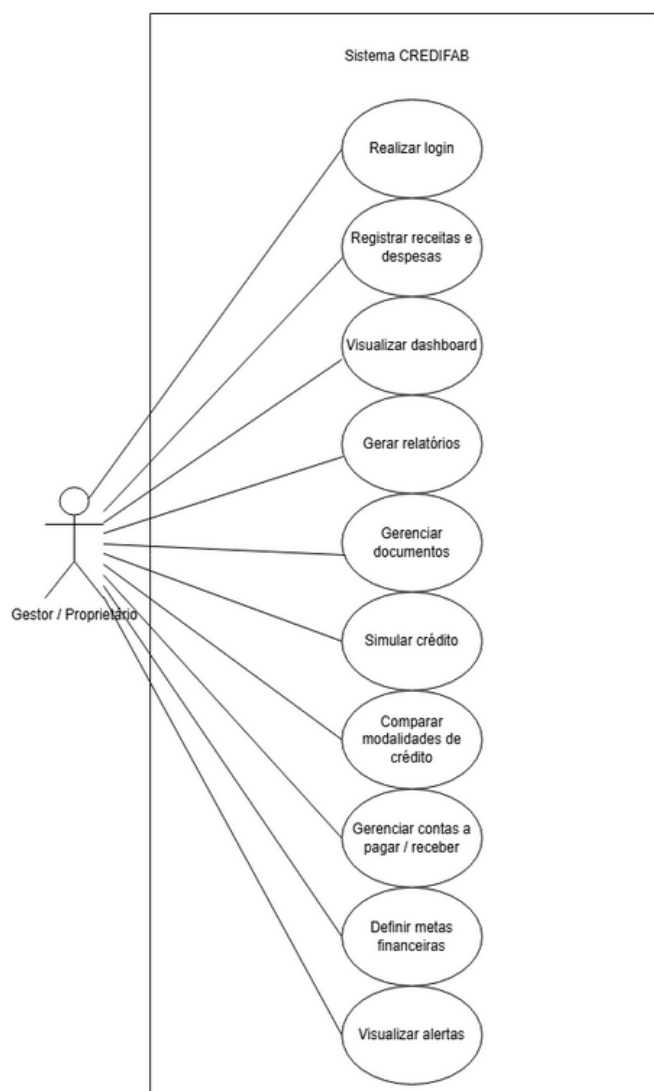
2.5.1.1 Diagrama dos Casos de Uso. O gestor/proprietário será o usuário principal da plataforma, visto que será responsável pelo gerenciamento financeiro da empresa e organização documental. Os casos de uso a seguir refletem integralmente os requisitos R01–R14 do Backlog do produto (Documento de Visão, Tabela 9):

- **Cadastrar usuário e empresa** (R01, R04): criar conta de gestor e vincular à empresa correspondente;
- **Realizar login** (R03): autenticar o usuário e receber token JWT para acesso às rotas protegidas;
- **Recuperar senha** (R05): solicitar redefinição de senha via e-mail com token temporário;
- **Excluir usuário e empresa** (R02): encerrar a conta, com cascata de exclusão configurada nas chaves estrangeiras;
- **Gerenciar categorias** (R06): criar, editar e excluir categorias de receita e despesa por empresa;
- **Registrar transações** (R07): cadastrar manualmente receitas e despesas vinculadas a categorias;
- **Consultar histórico de transações** (R08): listar movimentações com filtros por período, tipo, categoria e valor;
- **Visualizar dashboard** (R09): exibir saldo, receitas, despesas e indicadores visuais consolidados;

- **Gerenciar documentos** (R10): realizar upload e organização de documentos fiscais, contábeis e jurídicos;
- **Gerar relatórios financeiros** (R11): produzir relatórios em PDF para gestores e instituições financeiras;
- **Simular crédito** (R12): calcular parcelas e impacto financeiro a partir de valor, prazo, modalidade e taxa;
- **Comparar modalidades de crédito** (R13): visualizar diferentes linhas de crédito lado a lado.

No diagrama de casos de uso na Figura 2 é possível visualizar como funcionaria o sistema CREDIFAB em suas principais funcionalidades sob a perspectiva do Gestor/Proprietário.

Figura 2: Diagrama de casos de uso do sistema CREDIFAB.



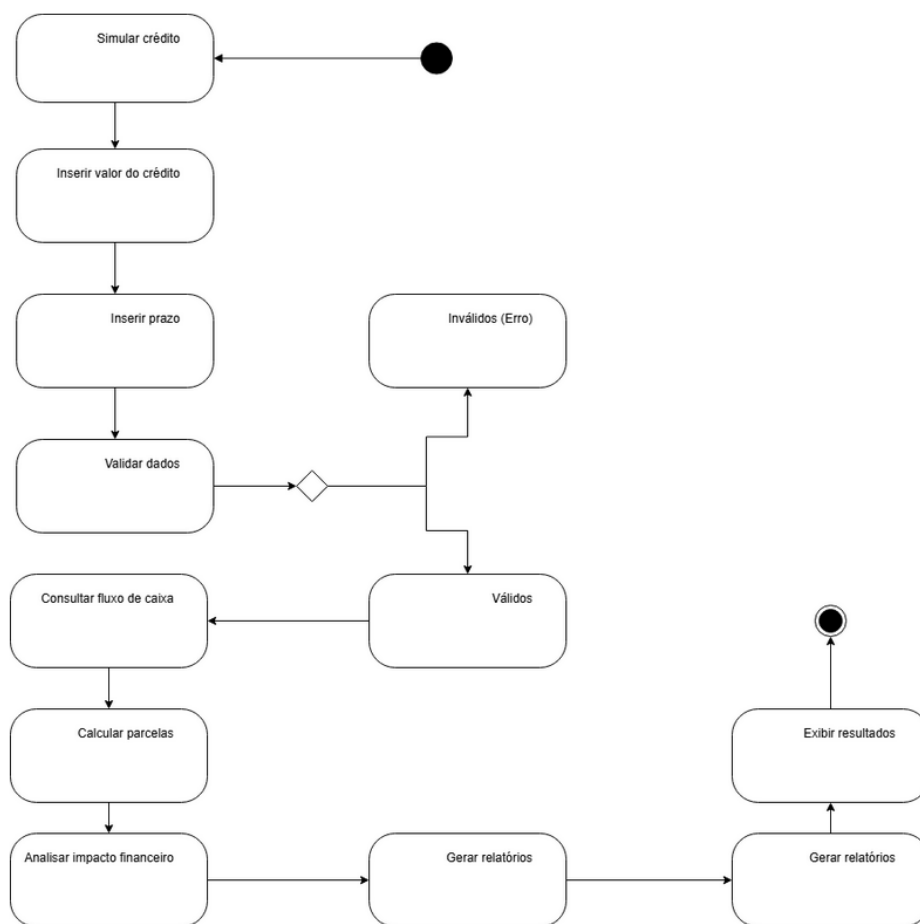
Fonte: elaborado pelos autores (2026).

Nota: O diagrama exibe os casos “Gerenciar contas a pagar/receber”, “Definir metas financeiras” e

“Visualizar alertas”, que não compõem o escopo atual (R01–R14) e serão removidos da imagem em revisão posterior. Em caso de divergência, prevalece a descrição textual desta seção.

2.5.1.2 Diagrama de atividades. A simulação de crédito, cujo objetivo é auxiliar o gestor na tomada de decisões financeiras, é a atividade mais complexa do sistema CREDIFAB devido à necessidade de validação de dados, execução de cálculos financeiros e análise do impacto das parcelas no fluxo de caixa da empresa. O processo é iniciado com a inserção dos parâmetros da simulação pelo usuário (valor, prazo, modalidade e taxa de juros), seguido pela validação das informações fornecidas. Após a validação, o sistema consulta o fluxo de caixa atual, realiza os cálculos das parcelas, avalia o impacto financeiro, gera o relatório consolidado e apresenta os resultados ao gestor. O diagrama de atividades (Figura 3) representa o fluxo dessa simulação.

Figura 3: Diagrama de atividades da simulação de crédito.



Fonte: elaborado pelos autores (2026).

Nota: A imagem apresenta a etapa “Gerar relatórios” de forma redundante; considera-se uma única geração de relatório consolidado no fluxo. Em caso de divergência, prevalece a descrição textual desta seção.

As funcionalidades essenciais do sistema CREDIFAB exigem uma arquitetura capaz de garantir segurança no processamento de dados, de suportar crescimento no número de usuários e no volume de dados, isso sem comprometer o desempenho da aplicação, e de separar as utilidades do sistema em camadas especializadas, onde cada componente possui atribuições específicas. Nesse sentido, é essencial adotar uma arquitetura Cliente-Servidor com uma comunicação via API RESTful, visto que trará ao frontend o objetivo da interação com o usuário, enquanto o backend processará as outras operações do sistema, como o cálculo financeiro. Além disso, o armazenamento de informações financeiras e documentos sensíveis demanda isolamento da camada de persistência e controle seguro de acesso, justificando o uso de serviços centralizados no servidor e autenticação baseada em JWT. Por fim, essa arquitetura possibilitará às equipes do frontend e do backend a trabalharem simultaneamente, o que facilita a manutenção, a evolução e a implantação do sistema em ambientes distribuídos.

2.5.2 Visão de organização lógica

A visão de organização lógica do CREDIFAB apresenta a divisão do sistema em módulos responsáveis por agrupar funcionalidades relacionadas ao domínio da aplicação. Essa organização foi definida para reduzir acoplamento, facilitar manutenção, permitir desenvolvimento paralelo entre os integrantes da equipe e favorecer a evolução incremental do produto ao longo das sprints. O sistema é organizado em módulos lógicos distribuídos principalmente entre o frontend React e o backend Flask. O frontend concentra os módulos de interface, navegação e comunicação com a API. O backend concentra os módulos de regras de negócio, autenticação, persistência e serviços de apoio. O acesso ao banco de dados ocorre exclusivamente pelo backend, por meio do ORM SQLAlchemy, evitando o acesso direto da interface aos dados persistidos. O **módulo de Autenticação e Usuários** é responsável pelo cadastro, login, recuperação de senha, exclusão de conta, geração e validação de tokens JWT. Ele protege as rotas privadas da aplicação e garante que apenas usuários autenticados acessem informações vinculadas à própria empresa. O **módulo de Empresa** centraliza os dados da organização cadastrada, servindo como referência para categorias, transações, documentos, simulações e relatórios. Essa decisão lógica permite isolar os dados de diferentes empresas e manter rastreabilidade das informações. O **módulo Financeiro** é responsável pelo registro de receitas, despesas e categorias, além do histórico de transações com filtros. Ele fornece os dados necessários para cálculo de fluxo de caixa, dashboard financeiro e relatórios. O **módulo de Dashboard e Relatórios** consolida informações financeiras em indicadores, gráficos e relatórios em PDF. Ele consome dados do módulo Financeiro e transforma essas informações em visualizações compreensíveis para o gestor. O **módulo de Crédito** é responsável pelas simulações de empréstimos, cálculo de parcelas, comparação de modalidades e análise do impacto financeiro no caixa da empresa. Por fim, o **módulo de Infraestrutura Compartilhada** reúne elementos

reutilizáveis da aplicação, como configurações, conexão com banco de dados, validações, tratamento de erros, middlewares de autenticação e padrões de resposta da API.

Figura 4: Tabela - Visão de organização lógica (Seção 2.5.2).

Módulo lógico	Responsabilidade	Principais interfaces / comunicação	Justificativa arquitetural
Interface do Usuário (Frontend)	Apresentar telas, formulários, dashboards e navegação da aplicação. Coletar entradas do usuário e exibir respostas da API.	Consome endpoints REST/JSON do backend via HTTPS.	Separa a camada de apresentação das regras de negócio, facilitando usabilidade e evolução da interface.
Autenticação e Usuários	Gerenciar cadastro, login, logout, hash de senha, emissão e validação de token JWT e controle de acesso.	Endpoints /auth; integração com middleware de autenticação e repositório de usuários.	Centraliza segurança e autenticação em um módulo coeso, reduzindo duplicidade de lógica.
Empresa	Manter dados cadastrais da empresa e vínculo entre usuário e organização.	Consumido pelos módulos Financeiro, Documentos, Relatórios, Crédito e Metas.	Garante isolamento de dados por empresa e rastreabilidade das operações.
Financeiro	Registrar receitas, despesas, categorias, histórico, contas a pagar e contas a receber, além de apoiar o cálculo do fluxo de caixa.	Endpoints financeiros; persistência em banco via SQLAlchemy; fornece dados para dashboard e relatórios.	Agrupar o núcleo funcional do sistema e concentra regras financeiras em um domínio específico.
Documentos	Realizar upload, organização, consulta e associação de documentos fiscais, contábeis e jurídicos à empresa.	Endpoints de documentos; interação com armazenamento e banco de metadados.	Isola a gestão documental, reduz acoplamento com o módulo financeiro e favorece controle de acesso.
Dashboard e Relatórios	Consolidar indicadores, gerar gráficos e produzir relatórios financeiros compreensíveis para o gestor.	Consome dados do módulo Financeiro e devolve respostas agregadas ao frontend.	Separa processamento analítico da entrada operacional de dados, melhorando manutenção e reuso.
Crédito	Executar simulações de crédito, cálculo de parcelas, comparação de modalidades e análise de impacto no caixa.	Consome dados financeiros e parâmetros informados pelo usuário; retorna cenários simulados.	Encapsula regras específicas de negócio relacionadas à preparação para crédito.
Metas e Alertas	Gerenciar metas financeiras e emitir alertas de vencimento, pendências e progresso.	Consulta transações, contas e metas; fornece notificações e status ao frontend.	Agrupar funcionalidades de acompanhamento contínuo e apoio à decisão do gestor.
Infraestrutura Compartilhada	Concentrar configurações, tratamento de erros, validações, conexão com banco, logs e componentes reutilizáveis.	Utilizado internamente por controllers, services e repositórios do backend.	Padroniza comportamentos transversais e melhora manutenibilidade, testes e consistência do sistema.

Fonte: elaborado pelos autores (2026).

Nota: A tabela na imagem ainda lista o módulo “Metas e Alertas” e itens de “contas a pagar/receber”, que ficaram fora do escopo atual. Valem os módulos descritos no texto desta seção. Em caso de divergência, prevalece a descrição textual desta seção.

2.5.3 Visão estrutural

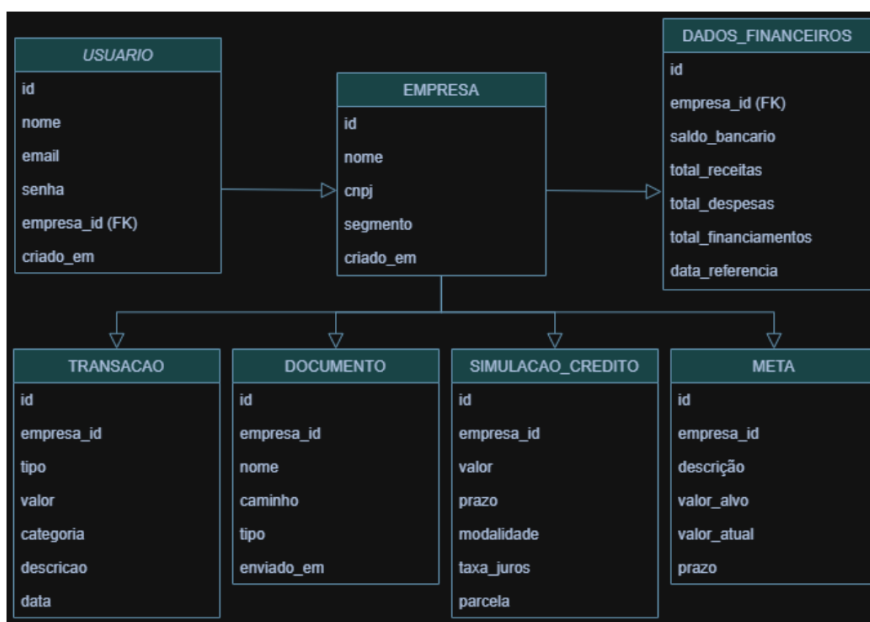
O sistema CREDIFAB é estruturado em três camadas — Frontend, Backend e Banco de Dados. Essa visão apresenta os elementos que compõem o sistema, suas responsabilidades e como se relacionam entre si, por meio dos diagramas de classes e de componentes.

2.5.3.1 Diagrama de Classes. O Diagrama de Classes, representado na Figura 5, apresenta as entidades do domínio do CREDIFAB e seus relacionamentos. A entidade central do sistema é a EMPRESA, à qual todas as demais estão associadas. As entidades são:

- **USUARIO:** representa o gestor ou proprietário que acessa a plataforma. Está diretamente vinculado a uma empresa por meio do atributo `empresa_id`. Possui campos como `nome`, `email`, `senha` (com hash) e `criado_em`.

- **EMPRESA:** representa a microempresa cadastrada na plataforma. É a entidade central do sistema — todas as movimentações, documentos e simulações estão associadas a ela. Possui campos como **nome**, **cnpj**, **segmento** e **criado_em**.
- **CATEGORIA:** representa uma classificação de receita ou despesa criada pela empresa (R06). Possui campos como **nome**, **tipo** (receita ou despesa) e **empresa_id**, com restrição de unicidade pelo par (**empresa_id**, **nome**).
- **TRANSACAO:** representa uma movimentação financeira registrada pela empresa, podendo ser uma receita ou despesa. Relaciona-se com **EMPRESA** (**empresa_id**) e com **CATEGORIA** (**categoria_id**), além dos campos **tipo**, **valor**, **descricao** e **data**.
- **DOCUMENTO:** representa um arquivo enviado pela empresa, como contratos, notas fiscais ou documentos contábeis, armazenando o caminho do arquivo, o tipo e a data de envio.
- **SIMULACAO_CREDITO:** representa o resultado de uma simulação realizada pelo gestor, contendo informações como valor solicitado, prazo, modalidade, taxa de juros e valor da parcela.

Figura 5: Diagrama de classes.



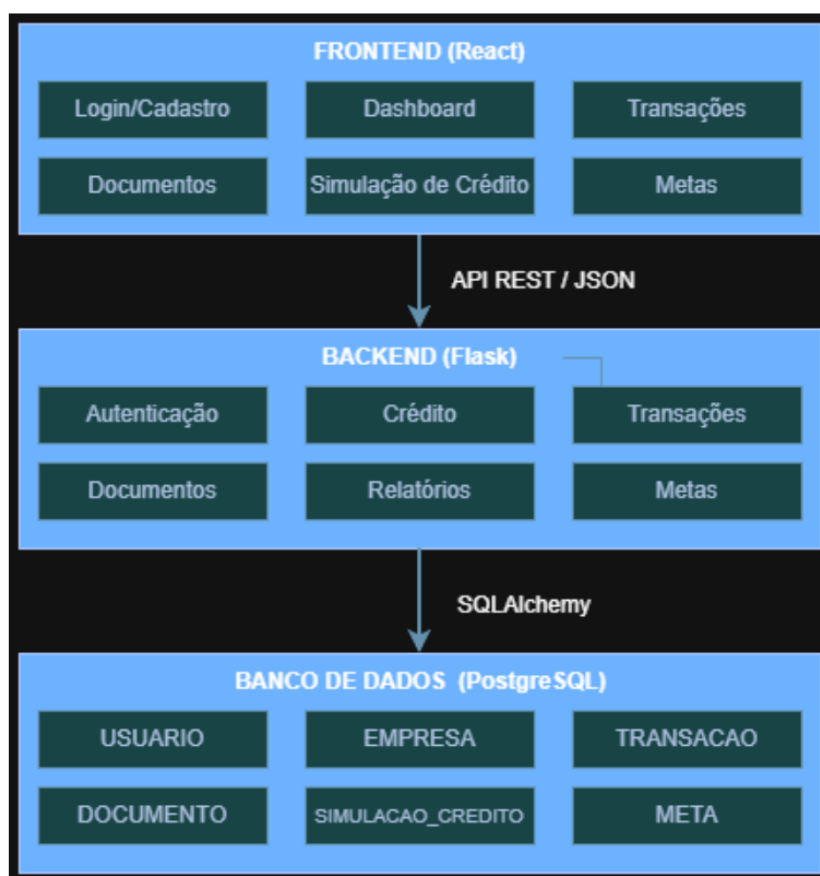
Fonte: elaborada pelos autores (2026).

Nota: A imagem ainda exibe as entidades DADOS_FINANCEIROS e META, fora do escopo atual, e não destaca a entidade CATEGORIA nem a FK categoria_id de TRANSACAO. Valem as seis entidades e relacionamentos descritos acima. Em caso de divergência, prevalece a descrição textual desta seção.

2.5.3.2 Diagrama de Componentes. O Diagrama de Componentes, apresentado na Figura 6, representa as três camadas do sistema CREDIFAB e seus respectivos módulos.

- **Frontend (React):** composto pelas telas de Login/Cadastro, Dashboard, Categorias, Transações, Documentos, Relatórios e Simulação de Crédito. Comunica-se com o Backend exclusivamente por meio de uma API REST, com troca de dados em formato JSON.
- **Backend (Flask):** concentra toda a lógica de negócio do sistema, organizada nos módulos de Autenticação, Categorias, Transações, Documentos, Relatórios e Crédito. Recebe as requisições do Frontend, processa as regras de negócio e acessa o Banco de Dados por meio do SQLAlchemy.
- **Banco de Dados (PostgreSQL):** responsável pelo armazenamento persistente de todos os dados do sistema, incluindo as tabelas USUARIO, EMPRESA, CATEGORIA, TRANSACAO, DOCUMENTO e SIMULACAO_CREDITO, acessado exclusivamente pelo Backend por meio do SQLAlchemy.

Figura 6: Diagrama dos componentes.



Fonte: elaborada pelos autores (2026).

Nota: A imagem ainda apresenta o módulo/tabela “Metas” no frontend, no backend e no banco, fora do

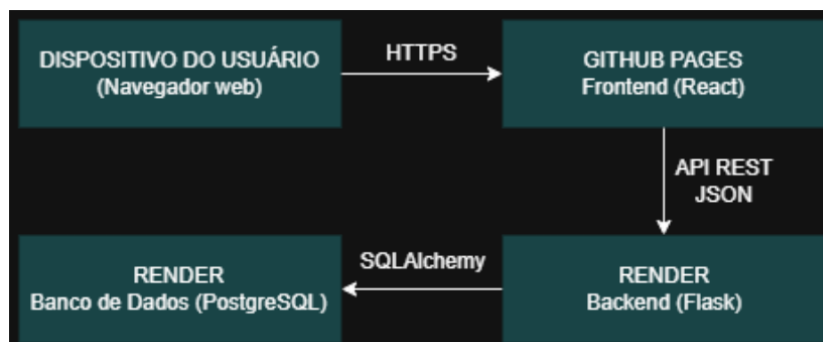
escopo atual; valem os módulos e tabelas listados no texto. Em caso de divergência, prevalece a descrição textual desta seção.

2.6 Visão de Implantação

O sistema CREDIFAB é implantado em uma arquitetura distribuída, com os componentes alocados em ambientes distintos, conforme representado na Figura 7.

- **Dispositivo do Usuário (Navegador Web):** ponto de acesso do sistema. O usuário acessa a plataforma diretamente pelo navegador, sem necessidade de instalação de software.
- **Vercel(Frontend):** responsável pela hospedagem da camada de apresentação desenvolvida em React.
- **Render — Backend (Flask):** responsável pela hospedagem da camada de lógica de negócio desenvolvida em Flask. Foi escolhido por oferecer suporte a aplicações Python, deploy automático via Git e HTTPS nativo.
- **Render — Banco de Dados (PostgreSQL):** responsável pelo armazenamento persistente dos dados do sistema. Foi escolhido por ser gerenciado pelo próprio Render, na mesma infraestrutura do Backend, minimizando a latência entre as camadas.

Figura 7: Diagrama de implantação.



Fonte: elaborada pelos autores (2026).

2.7 Restrições adicionais

O sistema CREDIFAB possui restrições adicionais relacionadas ao seu contexto de uso, ao escopo acadêmico do projeto, às tecnologias adotadas e à natureza sensível dos dados manipulados. A aplicação será acessível diretamente pela internet, por meio de navegador web, exigindo autenticação para acesso às funcionalidades internas. As únicas funcionalidades públicas serão cadastro, login e recuperação de senha. Todas as demais operações, como registro financeiro, consulta de relatórios, upload de documentos e simulação de crédito, exigirão usuário autenticado por token JWT, com o token transmitido no cabeçalho HTTP **Authorization** em cada requisição. Como o sistema manipula dados financeiros e documentos empresariais, cada usuário deverá acessar apenas as informações vinculadas à sua própria empresa. O backend será responsável por validar o vínculo entre usuário e empresa antes de retornar, alterar ou excluir qualquer dado. Essa restrição reduz o risco de acesso indevido entre empresas cadastradas. Por se tratar de um MVP acadêmico, o sistema utilizará dados fictícios ou simulados durante os testes e apresentações. Não deverão ser utilizados dados reais de empresas, documentos fiscais reais ou informações bancárias sensíveis. Essa restrição reduz riscos de privacidade e simplifica o processo de validação do produto. O CREDIFAB não realizará integração real com instituições financeiras, bancos, bureaus de crédito ou serviços externos de análise de risco nesta versão inicial. As simulações de crédito serão calculadas internamente a partir de parâmetros definidos pelo gestor, como valor solicitado, taxa de juros, prazo e modalidade. Essa decisão limita o escopo do projeto e torna a implementação compatível com o prazo da disciplina. Em relação à confiabilidade, o sistema deverá validar os dados informados pelo usuário antes de persistir qualquer informação. Valores financeiros, datas, tipos de transação, documentos enviados e parâmetros de simulação deverão ser verificados para reduzir inconsistências e evitar resultados incorretos nos relatórios e simulações. Em relação à portabilidade, a aplicação deverá funcionar em navegadores modernos e ser acessível sem instalação local. O frontend será publicado como aplicação web estática, enquanto o backend e o banco de dados serão configurados por variáveis de ambiente, facilitando implantação e manutenção em ambiente de nuvem. A disponibilidade do sistema dependerá dos serviços externos utilizados para hospedagem, como Vercel, Render e banco PostgreSQL gerenciado. Por utilizar recursos gratuitos ou de baixo custo no contexto acadêmico, não há garantia formal de alta disponibilidade, escalabilidade automática ou recuperação completa em caso de falhas externas. Essas restrições foram definidas para manter o sistema viável dentro do contexto da disciplina, proteger dados sensíveis, reduzir complexidade desnecessária e garantir que a arquitetura seja compatível com o escopo do MVP.

3 Bibliografia

- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.
- FOWLER, Martin. **Padrões de arquitetura de aplicações corporativas**. Tradução de Francisco Araújo. Porto Alegre: Bookman, 2006.
- MARTIN, Robert C. **Arquitetura limpa: o guia do artesão para estrutura e design de software**. Rio de Janeiro: Alta Books, 2019.
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.
- RICHARDS, Mark; FORD, Neal. **Fundamentals of software architecture: an engineering approach**. 1. ed. Sebastopol: O'Reilly Media, 2020.